Project 2

Sigurd Riseth
Department of Physics
University of Oslo
Oslo, Norway
sigurri@uio.no

***Abstract*—abstract.**

***Index Terms*—classification, regression, feed-forward neural networks, runge's function, MNIST**

I. Introduction

In the essay *The Bitter Lesson*, Richard Sutton [1] argues that the most significant advances in artificial intelligence have ultimately come from general methods that leverage computation, rather than approaches relying on extensive human-engineered domain knowledge. This observation highlights a central idea in modern machine learning: the ability to learn patterns and representations directly from data, rather than explicitly specifying how a problem should be solved. Artificial neural networks are one such approach, providing a flexible framework for learning complex relationships from data.

Linear and logistic regression provide relatively simple approaches to regression and classification, respectively. While these models offer interpretable and computationally efficient solutions, their predictive capabilities are constrained by their functional form. Neural networks extend this framework through compositions of parameterized transformations and nonlinear activation functions, allowing them to represent more complex relationships between inputs and outputs. This increased flexibility, however, also introduces challenges related to model architecture, optimization, and generalization.

The objective of this project is to implement and investigate feed-forward neural networks (FFNNs) for regression and classification. In particular, we study the effects of network architecture, activation functions, and optimization algorithms on model performance and training behavior. We also compare the resulting models with linear and logistic regression approaches, examining the trade-offs between model complexity, predictive performance, and computational cost.

The remainder of this report is structured as follows. Section II presents the theoretical background and methods used to construct and train the networks. Section III describes the datasets, preprocessing, implementation details, and verification of the code. Section IV presents and discusses the results, including comparisons with the linear and logistic regression models. Finally, Section V summarizes the main findings and outlines possible directions for future work. Appendix A documents our use of large language models. The code and results underlying the figures and tables are available in our publicly hosted repository [2].

II. Method

This section presents the cost functions and the gradient-based methods used to minimize them, then the models, linear regression, logistic regression, and neural networks, and finally the evaluation methods used. We assume the reader has familiarity with basic linear algebra and statistics.

A. Cost functions and their gradients

1) Mean squared error: For regression, we measure the fit of the predictions $\tilde{y}_i$ to the targets y_i by the mean squared error,

$$C_{\text{MSE}} = \frac{1}{n}\sum_{i=0}^{n-1}(y_i - \tilde{y}_i)^2\,, \tag{1}$$

the same quantity that OLS minimizes. Squaring the residuals penalizes large errors heavily, and up to constants the MSE is the negative log-likelihood of Gaussian noise as in eq. (9) [3]. Its derivative with respect to a prediction is

$$\frac{\partial C_{\text{MSE}}}{\partial \tilde{y}_i} = \frac{2}{n}(\tilde{y}_i - y_i)\,, \tag{2}$$

which is all that backpropagation needs from the cost. For the linear model $\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}$, the chain rule recovers $\nabla_{\boldsymbol{\theta}} C_{\text{MSE}} = \frac{2}{n}\boldsymbol{X}^\top(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y})$.

a) Penalties: Any of the costs in this section can be extended with an ℓ_2 or ℓ_1 penalty on the weight matrices $\boldsymbol{W}^l$ of the network,

$$C_{\ell_2} = C + \lambda\sum_l \|\boldsymbol{W}^l\|_2^2, \qquad C_{\ell_1} = C + \lambda\sum_l \|\boldsymbol{W}^l\|_1, \tag{3}$$

with the norms taken element-wise and $\lambda \geq 0$. The penalties add $2\lambda\boldsymbol{W}^l$ and $\lambda\,\text{sign}(\boldsymbol{W}^l)$ to the gradient, respectively. As for Lasso, the ℓ_1 penalty is not differentiable at zero, and we use $\text{sign}(0) = 0$, the minimal-norm subgradient. For the linear model, these are the Ridge and Lasso costs of section II-C.

2) Binary cross entropy:

3) Categorical cross entropy:

B. Gradient-based optimization

Apart from OLS and Ridge regression, none of the costs in this report have a closed-form minimizer, so we minimize them iteratively with gradient descent (GD). For a differentiable cost $C(\boldsymbol{\theta})$, GD iterates

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \gamma\nabla_{\boldsymbol{\theta}} C\big(\boldsymbol{\theta}^{(k)}\big), \tag{4}$$

with learning rate $\gamma > 0$, at the cost of one gradient evaluation per iteration [4].

a) Momentum: Momentum adds a fraction $\beta \in [0, 1)$ of the previous step to the current one,

$$\boldsymbol{v}^{(k)} = \beta \boldsymbol{v}^{(k-1)} + \gamma \nabla_{\boldsymbol{\theta}} C\big(\boldsymbol{\theta}^{(k)}\big), \qquad \boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \boldsymbol{v}^{(k)}, \tag{5}$$

with $\boldsymbol{v}^{(0)} = \mathbf{0}$. Steps along directions where the gradient keeps its sign accumulate, while oscillations across a narrow valley partly cancel, which helps on ill-conditioned costs [3], [4].

b) Adaptive learning rates: Adaptive methods give each parameter its own effective learning rate. AdaGrad [5] divides the step of each coordinate by the root of its accumulated squared gradients, so the effective step can only decrease. RMSprop replaces the sum by an exponentially decaying average,

$$\begin{aligned} \boldsymbol{v}^{(k)} &= \rho \boldsymbol{v}^{(k-1)} + (1-\rho)\, \boldsymbol{g}^{(k)} \odot \boldsymbol{g}^{(k)}, \\ \boldsymbol{\theta}^{(k+1)} &= \boldsymbol{\theta}^{(k)} - \frac{\gamma}{\sqrt{\boldsymbol{v}^{(k)}} + \epsilon} \odot \boldsymbol{g}^{(k)}, \end{aligned} \tag{6}$$

so the step size can recover [3]. Here $\boldsymbol{g}^{(k)}$ is the gradient, $\odot$ the Hadamard product, and the square root and division are element-wise. Adam [6] combines a decaying average of the gradient, as in momentum, with the scaling of RMSprop, and corrects both averages for their initialization at zero:

$$\begin{aligned} \boldsymbol{m}^{(k)} &= \beta_1 \boldsymbol{m}^{(k-1)} + (1-\beta_1)\, \boldsymbol{g}^{(k)}, \\ \boldsymbol{v}^{(k)} &= \beta_2 \boldsymbol{v}^{(k-1)} + (1-\beta_2)\, \boldsymbol{g}^{(k)} \odot \boldsymbol{g}^{(k)}, \\ \boldsymbol{\theta}^{(k+1)} &= \boldsymbol{\theta}^{(k)} - \gamma\, \frac{\boldsymbol{m}^{(k)}/(1-\beta_1^k)}{\sqrt{\boldsymbol{v}^{(k)}/(1-\beta_2^k)} + \epsilon}. \end{aligned} \tag{7}$$

We use $\rho = 0.9$ for RMSprop, $\beta_1 = 0.9$ and $\beta_2 = 0.999$ for Adam, and $\epsilon = 10^{-8}$. Since Adam divides by the running root-mean-square of the gradient, its steps do not shrink with the gradient near a minimum, and with a fixed γ it keeps taking steps of order γ there.

c) Stochastic gradient descent: For a cost that is an average over the data, $C(\boldsymbol{\theta}) = \frac{1}{n}\sum_i c_i(\boldsymbol{\theta})$, stochastic gradient descent (SGD) replaces the full gradient by the average over a mini-batch $\mathcal{B}_k$ of M points,

$$\boldsymbol{g}^{(k)} = \frac{1}{M} \sum_{i \in \mathcal{B}_k} \nabla_{\boldsymbol{\theta}} c_i\big(\boldsymbol{\theta}^{(k)}\big). \tag{8}$$

This is an unbiased estimate of the full gradient [3]. We shuffle the training set once per epoch and split it into $\lceil n_{\text{tr}}/M \rceil$ mini-batches, so one epoch costs the same number of floating-point operations as one full-batch step but makes $\lceil n_{\text{tr}}/M \rceil$ updates. The gradient noise does not vanish at the minimum, so with a fixed learning rate the iterates fluctuate around it; a decaying learning rate, here $\gamma_t = \gamma_0/(1+\delta t)$ or $\gamma_t = \gamma_0 e^{-\delta t}$ with t the number of updates, damps these fluctuations [4]. Any of the update rules above can be applied to $\boldsymbol{g}^{(k)}$.

d) Early stopping: Early stopping is a form of regularization that limits overfitting by terminating training when performance on a validation set ceases to improve [4]. While the training loss may continue to decrease, an increasing validation loss can indicate that the model is beginning to overfit the training data [7]. We therefore monitor the validation loss throughout training and stop if it has not decreased for five consecutive epochs. The model weights are subsequently restored to those obtained at the epoch with the lowest validation loss.

C. Linear regression

Given a dataset of n samples $\mathcal{D} = \{(y_i, x_i)\}_{i=0}^{n-1}$ and assuming that the targets y_i are generated by

$$\boldsymbol{y} = \boldsymbol{f} + \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \boldsymbol{I}_n), \tag{9}$$

where $\boldsymbol{f}$ collects the values of an unknown function f. A polynomial model of degree p can then be written as the prediction $\tilde{\boldsymbol{y}} = \theta_0 + \boldsymbol{X}\boldsymbol{\theta}$, with $\boldsymbol{X} \in \mathbb{R}^{n \times p}$ the design matrix containing the polynomials $\boldsymbol{x}, \ldots, \boldsymbol{x}^p$, and $\boldsymbol{\theta} \in \mathbb{R}^p$ the parameter vector, with θ_0 the intercept. The goal of linear regression is to choose $\hat{\boldsymbol{\theta}}$ so that the fitted model $g(\boldsymbol{X}; \hat{\boldsymbol{\theta}}) = \theta_0 + \boldsymbol{X}\hat{\boldsymbol{\theta}}$ best approximates the unknown function f on the data $\mathcal{D}$ [4], [8].

One solution to this is to minimize the mean squared error (MSE), which yields the ordinary least squares (OLS) predictor,

$$C_{\text{OLS}}(\boldsymbol{\theta}) = \frac{1}{n} \left\| \boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta} \right\|_2^2. \tag{10}$$

This cost is convex, since its Hessian $\boldsymbol{H} = \frac{2}{n}\boldsymbol{X}^\top \boldsymbol{X}$ is positive semi-definite, so every stationary point is a global minimizer. Setting the gradient $\nabla_{\boldsymbol{\theta}} C_{\text{OLS}} = \frac{2}{n}\boldsymbol{X}^\top(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y})$ to zero gives the normal equations; if $\boldsymbol{X}$ has full column rank, the normal equations have the unique solution

$$\hat{\boldsymbol{\theta}}_{\text{OLS}} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}. \tag{11}$$

By adding the squared ℓ_2-norm of the parameter vector as a penalty to the cost function, we obtain Ridge regression [9],

$$C_{\text{Ridge}}(\boldsymbol{\theta}) = \frac{1}{n} \left\| \boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta} \right\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2, \qquad \lambda \geq 0. \tag{12}$$

For $\lambda > 0$ the Hessian is positive definite, so the cost is strictly convex. This yields the minimizer

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}}(\lambda) = \left(\boldsymbol{X}^\top \boldsymbol{X} + n\lambda \boldsymbol{I}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{y}. \tag{13}$$

Lasso [10] instead applies an ℓ_1 penalty,

$$C_{\text{Lasso}}(\boldsymbol{\theta}) = \frac{1}{n} \left\| \boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta} \right\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1. \tag{14}$$

The cost is convex but not differentiable where a coefficient is zero, and it has no closed-form minimizer [8], so it is minimized with the methods of section II-B. Lasso performs variable selection while Ridge only shrinks [4].

D. Logistic regression

Logistic regression is a natural choice as a baseline. It is a simple model in which everything can be computed, its cost function is convex, and it is exactly a neural network with no hidden layers [4]. Its cost is the binary or categorical cross entropy of section II-A.

E. Neural networks

Linear and logistic regression share a common limitation: both act on a fixed design matrix $\boldsymbol{X}$, so the model can only be as expressive as the features we choose to put into it. A feed-forward neural network removes this limitation by inserting one or more *hidden layers* between the input and the output, each applying a linear map followed by a non-linear activation. The final layer is then nothing more than a linear or logistic regression, but it acts on features that are *learned* from the data rather than given in advance [4].

1) The artificial neuron: An artificial neuron computes a linear predictor, as in logistic regression, followed by an activation function. We number layers as $l = 0, 1, \ldots, L$, with $l = 0$ the input and $l = L$ the output, and M_l the number of nodes in layer L. For a single neuron j in layer l, the activation is

$$z_j^l = \sum_{i=1}^{M_{l-1}} w_{ij}^l a_i^{l-1} + b_j^l, \tag{15}$$

where w_{ij}^l is the weight connecting node i of layer $l-1$ to node j of layer l, and b_j^l is the bias of node j in layer l. We can then obtain the output by applying the activation function [4],

$$a_j^l = f(z_j^l). \tag{16}$$

This whole process is depicted in fig. 1, where the activation and output of a single neuron j in layer l is shown.

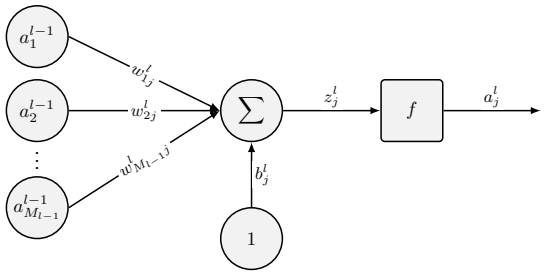


Fig. 1. The activation and output of a single neuron j in layer l, as shown in eqs. (15) and (16).

2) Forward pass: We can write the activation and output of all neurons the layer l in matrix-form, where eqs. (15) and (16) become

$$\boldsymbol{z}^l = (\boldsymbol{W}^l)^\top \boldsymbol{a}^{l-1} + \boldsymbol{b}^l, \qquad \boldsymbol{a}^l = \boldsymbol{f}(\boldsymbol{z}^l), \tag{17}$$

with $\boldsymbol{f}$ applied element-wise. Evaluating eq. (17) sequentially for all l-values is the feed-forward pass and returns the network output $\tilde{\boldsymbol{y}} = \boldsymbol{a}^L$ [4].

We can further collect the weights and biases into a single parameter vector $\boldsymbol{\Theta} = \{\boldsymbol{w}^l, \boldsymbol{b}^l\}_{l=1}^L$, and write the optimization problem as

$$\hat{\boldsymbol{\Theta}} = \arg\min_{\boldsymbol{\Theta}} C(\boldsymbol{\Theta}), \quad C(\boldsymbol{\Theta}) = \frac{1}{n}\sum_{i=1}^{n} L(y_i, \tilde{y}_i) + \lambda\Omega(\boldsymbol{\Theta}), \tag{18}$$

where L is the cost function and Ω is the regularization term [4]. This is a non-convex optimization problem, so there is no closed-form solution, and we must resort to the gradient-based methods reported in section II-B to find a local minimum.

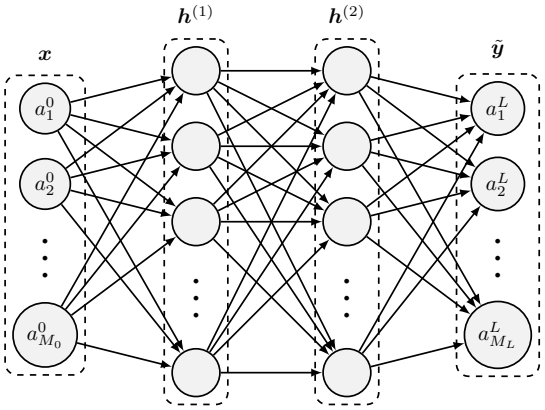


Fig. 2. Illustration of a fully connected feed-forward neural network. Adapted from Love [11].

3) Activation functions: The activation function must be non-linear and differentiable almost everywhere. If it were linear, the composition of multiple layers would collapse into a single affine transformation, and the network would be no more expressive than a single layer [4]. Our study will revolve around the following three activation functions:

a) Sigmoid: The logistic or sigmoid function is given by

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)(1-\sigma(z)),$$

and was the standard choice for decades. Its main drawback is that it saturates for large $|z|$, which leads to vanishing gradients and slow training [4].

b) ReLU: The rectified linear unit (ReLU),

$$\mathrm{ReLU}(z) = \max(0, z), \quad \mathrm{ReLU}'(z) = \begin{cases} 1, & \text{if } z > 0, \\ 0, & \text{if } z \leq 0, \end{cases}$$

is faster to compute, does not saturate for $z > 0$, and is a sound default choice for hidden layers. Its main drawback is that it can lead to dead neurons, which never activate and thus never learn [4].

The leaky ReLU (LReLU) repairs this issue by allowing a small gradient for negative values, so the gradient never completely vanishes. It is defined as

$$\text{Leaky ReLU}(z) = \begin{cases} z, & \text{if } z > 0, \\ \alpha z, & \text{if } z \leq 0, \end{cases} \quad \alpha \approx 0.01,$$

and its derivative is

$$\text{Leaky ReLU}'(z) = \begin{cases} 1, & \text{if } z > 0, \\ \alpha, & \text{if } z \leq 0. \end{cases}$$

4) Backpropagation: We can compute the gradient of the cost function with respect to the parameters using the back-propagation algorithm [12]. The key idea is to compute the error at the output layer and then propagate it backward through the network.

The error is given by

$$\boldsymbol{\delta}^l = (\boldsymbol{W}^{l+1}\boldsymbol{\delta}^{l+1}) \odot \boldsymbol{f}'(\boldsymbol{z}^l),$$

where $\odot$ denotes the element-wise product, and $\boldsymbol{\delta}^l$ is the error at layer l. For $l = L$, we don't have a next layer, but for the

two natural pairings of output layer and loss function (i.e., linear output with MSE loss and sigmoid output with binary cross-entropy loss), we have $\boldsymbol{\delta}^L = \boldsymbol{a}^L - \boldsymbol{y}$ [4]. The gradients with respect to the weights and biases are then given by

$$\frac{\partial C}{\partial \boldsymbol{W}^l} = \boldsymbol{\delta}^l \boldsymbol{a}^{l-1}, \qquad \frac{\partial C}{\partial \boldsymbol{b}^l} = \boldsymbol{\delta}^l.$$

Finally, we can update the parameters using gradient descent:

$$\boldsymbol{W}^l \leftarrow \boldsymbol{W}^l - \gamma \frac{\partial C}{\partial \boldsymbol{W}^l}, \qquad \boldsymbol{b}^l \leftarrow \boldsymbol{b}^l - \gamma \frac{\partial C}{\partial \boldsymbol{b}^l},$$

where γ is the learning rate.

5) Choice of output layer and loss function:

6) Weight initialization:

```
if initialization == "he":
    std = np.sqrt(2.0 / n_inputs)
elif initialization in ("xavier", "glorot"):
    std = np.sqrt(2.0 / (n_inputs + n_neurons))
```

7) Regularization: A network with more parameters than data points available can fit the entire training set exactly, so applying regularization is essential [4].

F. Evaluation

For regression, we evaluate a prediction $\tilde{\boldsymbol{y}}$ of targets $\boldsymbol{y}$ by

$$\mathrm{MSE} = \frac{1}{n}\sum_i (y_i - \tilde{y}_i)^2, \quad R^2 = 1 - \frac{\sum_i (y_i - \tilde{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \quad (19)$$

with $\bar{y}$ the mean of $\boldsymbol{y}$. An unbiased model with no variance reaches $\mathrm{MSE} = \sigma^2$ in expectation; R^2 compares against predicting $\bar{y}$ and is negative for a model that does worse than that constant [4].

III. Implementation

All methods were implemented from scratch in Python using NumPy [13]. The source code, unit tests, and experiment scripts are available in the project repository [2]. This section describes the datasets and preprocessing, the organization of the implementation, and the procedures used to verify correctness.

A. Data and preprocessing

For the regression task, we use Runge's function [14], a classical example for studying the bias–variance trade-off. It is defined as

$$f(x) = \frac{1}{1 + 25x^2}, \qquad x \in [-1, 1]. \quad (20)$$

As observed in Project 1 [15], polynomial regression can exhibit substantial oscillations near the endpoints of this interval. We generate $n = 100$ input samples uniformly from $[-1, 1]$, evaluate the function at these inputs, and add i.i.d. Gaussian noise with standard deviation $\sigma = 0.1$ to obtain the observed targets.

For the classification task, we use the MNIST dataset [16], which contains 70 000 grayscale images of handwritten digits, each of size 28×28 pixels, with labels corresponding to the ten classes $0, \ldots, 9$, split into 60 000 training and 10 000 test images. We flatten each image to a vector of 784 features scaled to $[0, 1]$, and set aside a stratified validation set of 10 000 training images. All hyperparameters are selected on the validation set; the test set is used only once, for the final comparison of the selected models.

B. Code structure

We reuse the structure and the gradient machinery of the previous project [15], which is summarized in algorithm 1. The code is structured in a package `fys_stk4155_p2`, containing the directory `nn`; which holds the code for the models including activation functions, layers (`Dense`, `Linear`, etc.), the network class `FFNN`, and loss functions.

Algorithm 1 Gradient descent with a generic update rule

Require: $\boldsymbol{\theta}^{(0)}$, schedule γ_t, batch size M ($M = n_{\text{tr}}$: full batch), epochs E, tolerance τ, update rule $\mathcal{U}$ with state $\boldsymbol{s}$

$t \leftarrow 0$, $\boldsymbol{s} \leftarrow \mathbf{0}$
for $e = 1, \ldots, E$ **do**
  Shuffle the training rows; split into $\lceil n_{\text{tr}}/M \rceil$ batches
  for each batch $\mathcal{B}$ **do**
    $\boldsymbol{g} \leftarrow \frac{1}{|\mathcal{B}|}\sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\theta}} c_i(\boldsymbol{\theta}^{(t)})$ ▷ analytical or JAX
    if $M = n_{\text{tr}}$ **and** $\|\boldsymbol{g}\|_\infty < \tau$ **then**
      return $\boldsymbol{\theta}^{(t)}$
    end if
    $(\boldsymbol{\theta}^{(t+1)}, \boldsymbol{s}) \leftarrow \mathcal{U}(\boldsymbol{\theta}^{(t)}, \boldsymbol{g}, \gamma_t, \boldsymbol{s})$ ▷ e.g. eqs. (4) and (7)
    $t \leftarrow t + 1$
  end for
end for
return $\boldsymbol{\theta}^{(t)}$

Thin constructor of the `FFNN` for logistic regression, which returns a single layer with softmax activation and cross entropy loss. This allows us to use this as a baseline model when evaluating the later neural networks.

C. Hyperparameter search

D. Verification

The developed package has a test suite which checks the code against closed-form results and independent implementations:

-

IV. Results and Discussion

This section presents and discusses the results from our experiments. First we cover the regression task, comparing linear regression to regression using FFNNs; afterward we cover the classification task of classifying handwritten digits using neural networks. Finally, we evaluate the methods critically across both tasks.

A. Regression

1) Sigmoid networks: We compare the best OLS model from Project 1 with two simple FFNN architectures; the resulting predictions are shown in fig. 3. The OLS model exhibits pronounced oscillations near the boundaries of the input domain, whereas both neural networks provide smoother

approximations of the underlying function. This illustrates one limitation of polynomial regression: its predictions can become unstable near the boundaries, particularly when using higher-degree polynomials. In contrast, the FFNNs learn the function directly from the input features, without requiring us to specify a polynomial design matrix. The input to the neural networks is therefore simply the one-dimensional feature matrix $\boldsymbol{X} \in \mathbb{R}^{n\times 1}$.

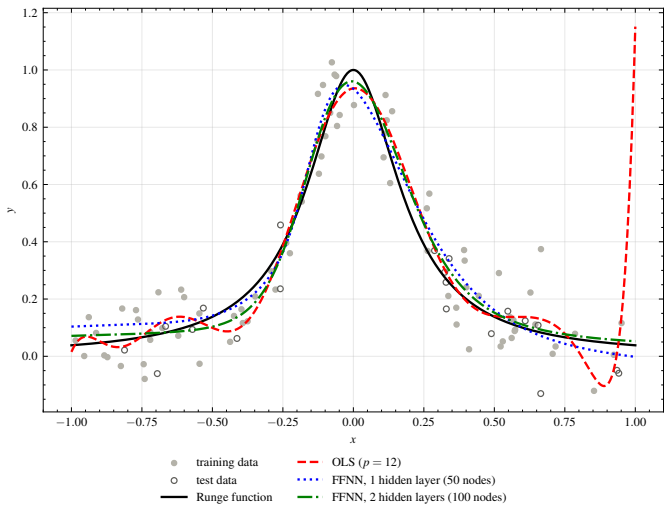


Fig. 3. Comparison of OLS and two FFNN architectures when approximating Runge's function. The data consist of 100 noisy samples with $\sigma = 0.1$, with 20% reserved for testing.

The quantitative results are presented in table I. The FFNN with two hidden layers of 100 neurons each achieves the lowest MSE relative to the true function, outperforming the one-hidden-layer FFNN and OLS by factors of roughly 1.7 and 10, respectively.

TABLE I
TRAINING AND TEST PERFORMANCE OF OLS AND TWO FFNN ARCHITECTURES ON RUNGE'S FUNCTION.

Model	Train MSE	Test MSE	Train R^2	Test R^2	MSE vs f
OLS ($p = 12$)	0.0081	0.0086	0.9207	0.6280	0.0125
FFNN (50)	0.0097	0.0092	0.9053	0.5996	0.0020
FFNN (100, 100)	0.0089	0.0085	0.9136	0.6315	0.0012

Interestingly, OLS achieves the lowest training MSE, but this does not translate into the best approximation of the underlying function. The discrepancy between its training and test performance, together with the oscillations visible in fig. 3 and high MSE vs f score, suggests that the polynomial model is sensitive to the noise in the training data and exhibits overfitting. The FFNN models provide a better approximation of the true function, despite both having a slightly higher training MSE.

The cost of choosing a larger neural network is shown in fig. 4, where we can see how the larger model gets stuck at a plateau around 0.1, matching the variance of the training targets (0.103), for a large period of the training. At this plateau, the model is predicting the mean of the data, and the gradient pointing towards useful curved features is tiny. This is the well-known plateau behavior of sigmoid networks [17], which can be avoided by using ReLU, or its variants, or applying batch normalization. **[HOW TRUE IS THE "COST" PART??? THEY USE DIFFERENT LEARNING RATES, OPTIMIZERS AND EPOCH COUNTS!!!]**

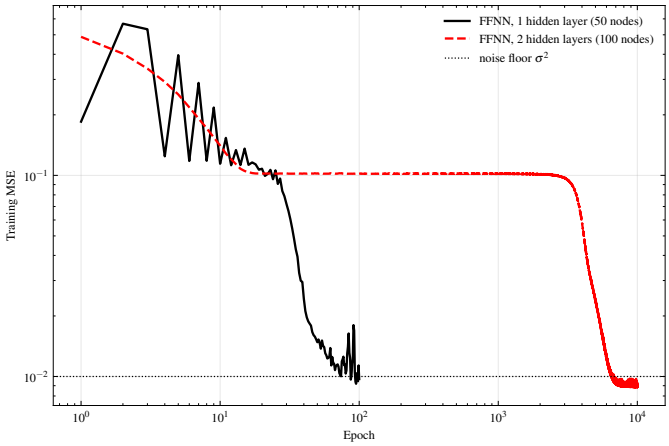


Fig. 4. Training MSE per epoch for the selected FFNNs: one hidden layer of 50 nodes (Adam, $\gamma = 0.1$, 100 epochs) and two hidden layers of 100 nodes (RMSProp, $\gamma = 10^{-4}$, 10000 epochs), both with a batch size of 32.

Meanwhile, in fig. 5, we can see how Adam and RMSProp escape much sooner than plain gradient descent at the same learning rate. Additionally, a higher learning rate escapes the plateau faster and reaches the optimum faster but with noticeably more noise. This motivates learning rate scheduling, which we will use in later parts of this project.

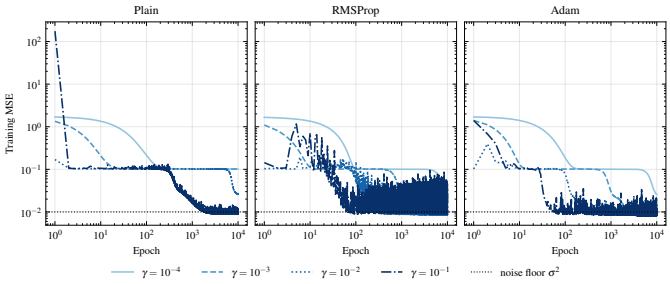


Fig. 5. Training MSE per epoch for the network with one hidden layer of 50 nodes, trained with plain SGD, RMSProp and Adam at batch size 16 for four learning rates.

2) The ReLU-family, larger networks and overfitting: Now we explore how the networks perform as a function of depth and width, including when substituting the hidden layers' activation functions with ReLU and its leaky variant.

In fig. 6 we observe how the 5-fold CV MSE generally increases with depth, particularly at eight hidden layers, and that the ReLU family degrades more quickly than the sigmoid. Shallow networks perform best for all three activations, and no configuration diverged.

Table II shows how each activation function's best fit aligns with Runge's function. ReLU-family networks are able to

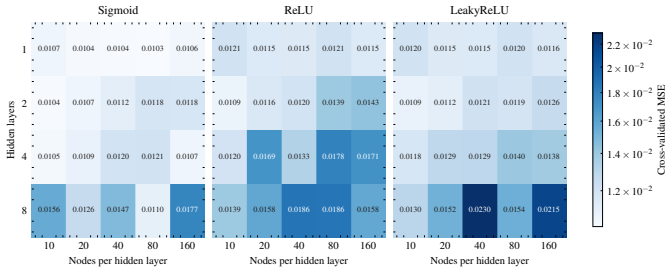


Fig. 6. Cross-validated MSE for each combination of activation function, number of hidden layers and nodes per hidden layer. Each cell shows the lowest 5-fold MSE over the learning rates $\gamma \in \{10^{-2}, 10^{-3}, 10^{-4}\}$, with all networks trained using Adam for 5000 epochs with a batch size of 32. All panels share the same logarithmic color scale; lighter is better.

reach lower training loss but slightly higher test loss than sigmoid networks, consistent with fitting noise **[WAY TOO SMALL DIFFERENCES!?]**. In fig. 7, we can also observe how the ReLU networks are piecewise linear with only a few visible kinks, so they approximate the peak and the tails with straight segments. Sigmoids naturally give smooth curves, which may suit Runge's function better.

TABLE II
TRAINING AND TEST PERFORMANCE OF THE ARCHITECTURE WITH THE LOWEST CROSS-VALIDATED MSE FOR EACH ACTIVATION FUNCTION.

Activation	Train MSE	Test MSE	Test R^2	MSE vs f
Sigmoid (1×80)	0.0091	0.0076	0.6684	0.0015
ReLU (2×10)	0.0083	0.0080	0.6513	0.0022
Leaky ReLU (2×10)	0.0083	0.0080	0.6504	0.0021

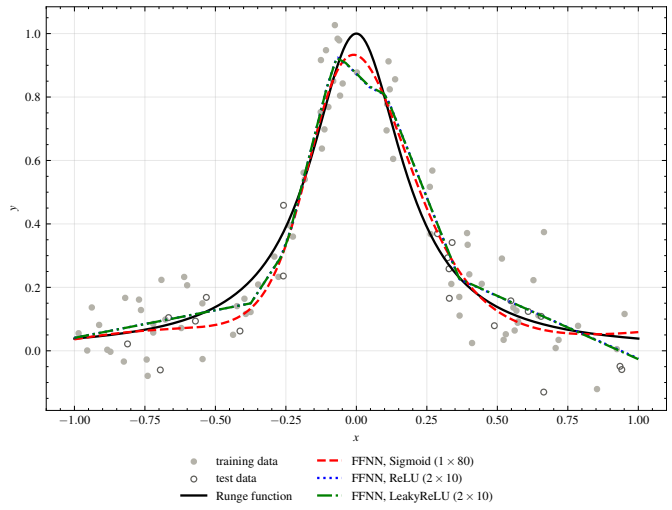


Fig. 7. Predictions of the best sigmoid, ReLU and Leaky ReLU networks from table II, together with the Runge function and the noisy training and test data. The ReLU and Leaky ReLU curves are nearly identical and consist of a few straight segments, while the sigmoid curve is smooth.

3) Adding regularization:

B. Image classification

1) Logistic regression: To judge whether hidden layers are worth their cost, we first need a baseline. Multinomial logistic regression is the FFNN with no hidden layers: a single dense layer maps the 784 pixels to 10 logits, followed by the softmax and the cross entropy. The model cannot learn features of its own, so each class score is a weighted sum of the raw pixel values.

The selected model reaches a test accuracy of 92.6% (table IV), in line with the roughly 92% expected for logistic regression on raw pixels, and far above the 11.4% obtained by always predicting the most common digit.

The accuracy hides significant differences between the digits. The confusion matrix in fig. 8 shows a recall of about 97% for 0 and 1, but only 87% for 5 and 89% for 2, with most errors between digits that share strokes, such as 5 and 3. Because the model is linear, each column of the weight matrix can be viewed as a 28×28 template (fig. 9). These templates are tied to fixed pixel positions, so a digit that is shifted or slanted matches the wrong template. Learning features that handle this kind of variation is what we expect hidden layers to do.

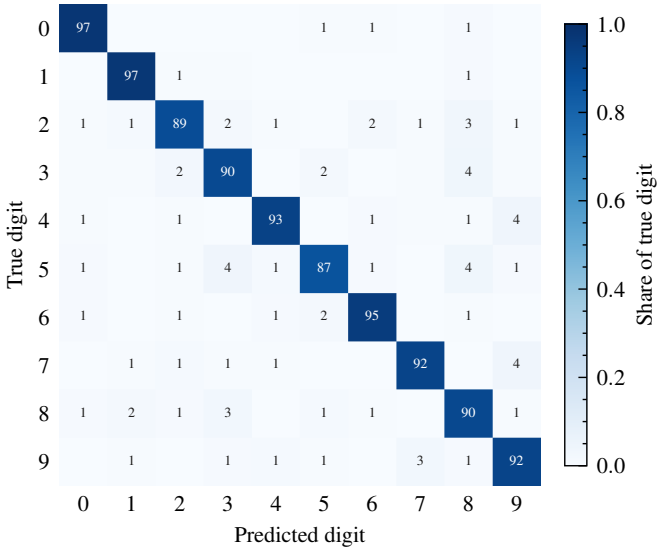


Fig. 8. Confusion matrix of the selected logistic regression model on the validation set, normalized by row so that the diagonal shows the recall of each digit (%).

2) Neural networks: For the FFNN, we use our validation set for model selection and tuning of hyperparameters. The search space defined in table III was searched for 10 epochs and the best performing model on the validation set was selected. We see that the best performing model is the largest possible ReLU network, with a moderate learning rate, allowing the network to capture complex features of the images. This also illustrates a weakness of our experiment, as an even larger model might have performed even better.

The hyperparameter search's various architecture performances are summarized in fig. 10, where each combination of activation function and number of hidden layers and nodes

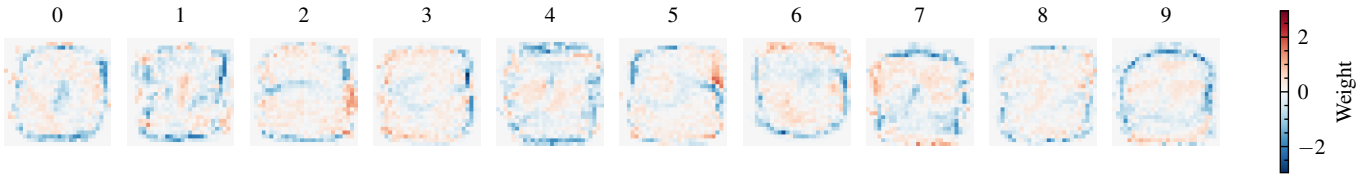


Fig. 9. Weights of the logistic regression model for each digit, reshaped to 28×28. Red pixels increase the score for the digit and blue pixels decrease it.

TABLE III
CLASSIFICATION FFNN HYPERPARAMETER SEARCH SPACE AND SELECTED CONFIGURATION BASED ON VALIDATION PERFORMANCE.

Hyperparameter	Search space	Selected
Number of layers	$\{4, 8\}$	8
Nodes per layer	$\{40, 80, 160\}$	160
Learning rate (γ)	$\{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$	10^{-3}
Activation function	Sigmoid, ReLU, Leaky ReLU	ReLU
ℓ_2 regularization (λ)	$\{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 0\}$	0

has its best validation accuracy and the learning rate which gave it listed. Here we see that both ReLU and Leaky ReLU prefer the larger networks, but the gains are marginal (0.5% validation accuracy gain from 4×40 vs. 8×160).

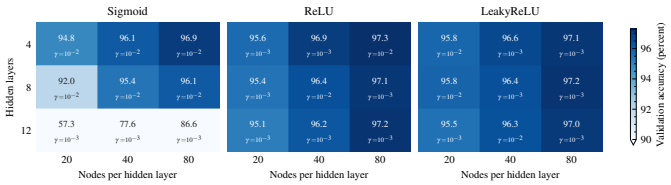


Fig. 10. .

The final model was retrained for 30 epochs, tracking the validation cost for identifying potential overfitting.

TABLE IV
TEST ACCURACY AND MACRO-AVERAGED F1 SCORE OF THE CLASSIFICATION MODELS.

Model	Accuracy	Macro-averaged F1
Majority class	0.1135	
Logistic regression	0.9261	
FFNN	0.9786	0.9783

C. Critical evaluation

V. CONCLUSION

TODO: conclusion.
Findings, limitations, future work, LLM declaration

REFERENCES

[1] R. S. Sutton, *The bitter lesson*, Mar. 2019. [Online]. Available: http://www.incompleteideas.net/IncIdeas/BitterLesson.html

[2] S. Riseth, *FYS-STK4155 Project 2*, version v0.1.0, GitHub, Nov. 2026. [Online]. Available: https://github.com/sigurdriseth/FYS-STK4155-P2

[3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. Accessed: Aug. 12, 2026. [Online]. Available: https://www.deeplearningbook.org/

[4] M. Hjorth-Jensen, *Machine Learning and the Physical Sciences: From Discriminative Methods to Generative Methods and Reinforcement Learning*. Springer, Oct. 9, 2026. [Online]. Available: https://github.com/EducationalMaterialUiO / MachineLearningUiO / blob / main/doc/BookML/Lecturebook.pdf

[5] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," *Journal of Machine Learning Research*, **12**, no. 61, pp. 2121–2159, 2011. [Online]. Available: http://jmlr.org/papers/v12/duchi11a.html

[6] D. P. Kingma and J. Ba. "Adam: A Method for Stochastic Optimization." arXiv: 1412.6980 [cs.LG], Accessed: Sep. 21, 2026. [Online]. Available: http://arxiv.org/abs/1412.6980 pre-published.

[7] A. Géron, *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems* (Data Science / Machine Learning), Third edition. O'Reilly, 2023, 834 pp., ISBN: 978-1-0981-2597-4.

[8] P. Mehta et al., "A high-bias, low-variance introduction to Machine Learning for physicists," *Physics Reports*, **810**, pp. 1–124, May 2019, ISSN: 03701573. DOI: 10.1016/j.physrep.2019.03.001 Accessed: Aug. 26, 2026. [Online]. Available: http://arxiv.org/abs/1803.08823

[9] A. E. Hoerl and R. W. Kennard, "Ridge Regression: Biased Estimation for Nonorthogonal Problems," *Technometrics*, **12**, no. 1, pp. 55–67, Feb. 1, 1970, ISSN: 0040-1706. DOI: 10.1080/00401706.1970.10488634 Accessed: Sep. 22, 2026. [Online]. Available: https://doi.org/10.1080/00401706.1970.10488634

[10] R. Tibshirani, "Regression Shrinkage and Selection Via the Lasso," *Journal of the Royal Statistical Society: Series B (Methodological)*, **58**, no. 1, pp. 267–288, Jan. 1, 1996, ISSN: 0035-9246. DOI: 10.1111/j.2517-6161.1996.tb02080.x Accessed: Sep. 22, 2026. [Online]. Available: https://doi.org/10.1111/j.2517-6161.1996.tb02080.x

[11] F. Love, *NNTikZ: TikZ Diagrams for Deep Learning and Neural Networks*, 2024. Accessed: Oct. 15, 2026. [Online]. Available: https://github.com/fraserlove/nntikz

[12] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, **323**, no. 6088, pp. 533–536, Oct. 1986, ISSN: 1476-4687. DOI: 10.1038/323533a0 Accessed: Oct. 27, 2026. [Online]. Available: https://www.nature.com/articles/323533a0

[13] C. R. Harris et al., "Array programming with NumPy," *Nature*, **585**, no. 7825, pp. 357–362, Sep. 17, 2020, ISSN: 0028-0836, 1476-4687. DOI: 10.1038/s41586-020-2649-2 Accessed: Aug. 23, 2026. [Online]. Available: https://www.nature.com/articles/s41586-020-2649-2

[14] C. Runge, "Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten," *Zeitschrift für Mathematik und Physik*, **46**, pp. 224–243, 1901.

[15] S. Riseth, *FYS-STK4155 Project 1*, version v0.1.0, GitHub, Oct. 2026. [Online]. Available: https://github.com/sigurdriseth/FYS-STK4155-P1

[16] Y. LeCun, C. Cortes, and C. J. Burges, "MNIST handwritten digit database," *ATT Labs [Online]. Available: http://yann.lecun.com/exdb/mnist*, **2**, 2010.

[17] A. M. Saxe, J. L. McClelland, and S. Ganguli. "Exact solutions to the nonlinear dynamics of learning in deep linear neural networks." arXiv: 1312.6120 [cs.NE], Accessed: Nov. 2, 2026. [Online]. Available: http://arxiv.org/abs/1312.6120 pre-published.

APPENDIX

A. Use of AI/LLM tools

Large language models were used throughout this project, following the course guidelines[1], which also define the contribution levels below. All code was tested, all derivations were checked, and every claim in the report was read, understood and verified by the author.

1) Tools used:

- TODO: tools, models, and dates.

TODO: summary of how each tool was used.

2) Text writing and editing: Table V gives the contribution level for each part of the report.

TABLE V
LLM CONTRIBUTION TO TEXT WRITING AND EDITING (LEVELS 0–3).

Section	Level	Notes
Abstract	TODO	
Introduction	TODO	
Methods	TODO	
Implementation	TODO	
Results and discussion	TODO	
Conclusion	TODO	
Appendices	TODO	

3) Code generation and assistance: Table VI gives the level for each source file (levels 0–4). Functions with level 2 or above carry an `LLM-assisted` tag in their docstring.

[1] https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/LLM_Usage_Declaration_Guidelines.md

4) Declared and undeclared use: TODO: one example of a use that must be declared, and one example of a use that need not be declared (e.g. using an LLM as a tutor and then writing about it in your own words).

B. Additional results

TODO: additional results.

TABLE VI
LLM CONTRIBUTION TO THE CODE (LEVELS 0–4).

File	Level	Description
`regression.ipynb`	3	Refactoring code and markdown cells for readability and write the code for producing fig. 3. Parallelize the CV search for optimal hyperparameters.
`classification.ipynb`	...	...